BỘ GIÁO DỤC VÀ ĐÀO TẠO

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI**

# BÁO CÁO BÀI THỰC HÀNH LAB 3

## ĐỀ 2: HỆ THỐNG GIÁM SÁT MÔI TRƯỜNG VƯỜN CÂY CÔNG NGHIỆP
## ỨNG DỤNG ESP32 VÀ FREERTOS

Sinh viên thực hiện: Bùi Đức Mạnh

Mã sinh viên: 231432289

Lớp: Hệ thống nhúng (N02)

Giảng viên hướng dẫn: Thầy Võ Quang Sơn

Đường dẫn GitHub: https://github.com/ManhPro1003/Bui-Duc-Manh_231432289_De-2_N02.git

Đường dẫn mô phỏng Wokwi: https://wokwi.com/projects/480205646642266113

Video thuyết trình mô phỏng: https://drive.google.com/drive/folders/1SYdPJ18-YBDyV55R6eJEW_ZYFvUTiCvM?usp=drive_link

Hà Nội – 2026

# MỤC LỤC

MỤC LỤC ..... 2
LỜI MỞ ĐẦU ..... 3
CHƯƠNG 1. RTOS VÀ CÁC CƠ CHẾ GIAO TIẾP GIỮA CÁC TASK ..... 4
1.1. Giới thiệu RTOS và ưu điểm ..... 4
1.2. FreeRTOS: ưu điểm và hạn chế ..... 5
1.3. Cơ chế giao tiếp giữa các Task ..... 6
1.3.1. Queue ..... 6
1.3.2. Semaphore và Mutex ..... 7
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG ..... 9
2.1. Mô hình tổng thể ..... 9
2.2. Linh kiện và sơ đồ kết nối ..... 10
2.3. Nguyên lý hoạt động ..... 11
CHƯƠNG 3. TRIỂN KHAI FREERTOS TRÊN ESP32 ..... 12
3.1. Bảng thông tin các Task ..... 12
3.2. Khởi tạo Queue và Mutex ..... 12
3.3. Thuật toán điều khiển máy phun sương ..... 13
CHƯƠNG 4. GIAO TIẾP MQTT VÀ THIẾT KẾ WEBSITE ..... 15
4.1. Kết nối Wi-Fi và MQTT ..... 15
4.2. Cấu trúc bản tin JSON ..... 15
4.3. Thiết kế Website ..... 15
CHƯƠNG 5. KẾT QUẢ THỰC HIỆN VÀ KIỂM THỬ ..... 17
5.1. Kết quả giám sát và truyền dữ liệu ..... 17
5.2. Kết quả điều khiển thiết bị ..... 19
CHƯƠNG 6. CODE VÀ PHÂN TÍCH CODE ..... 24
6.1. Link code ..... 24
6.2. Phân tích code ..... 24
6.2. Quá trình kết nối và cập nhật dữ liệu ..... 27
6.3. Quá trình điều khiển và cập nhật trạng thái thiết bị ..... 31
KẾT LUẬN ..... 37
TÀI LIỆU THAM KHẢO ..... 38

# LỜI MỞ ĐẦU

Trong bối cảnh công nghệ Internet vạn vật (IoT) ngày càng phát triển, việc ứng dụng các hệ thống giám sát và điều khiển tự động vào nông nghiệp đang trở thành một hướng đi có nhiều tiềm năng. Đặc biệt, đối với các loại cây công nghiệp như cà phê và hồ tiêu, việc theo dõi các yếu tố môi trường như nhiệt độ, độ ẩm và điều khiển thiết bị phù hợp có ý nghĩa quan trọng trong quá trình chăm sóc cây trồng.

Xuất phát từ kiến thức đã được học và sự hướng dẫn tận tình của **Thầy Võ Quang Sơn**, em thực hiện đề tài thiết kế hệ thống giám sát môi trường vườn cây công nghiệp sử dụng ESP32, hệ điều hành thời gian thực FreeRTOS và giao thức truyền thông MQTT. Hệ thống có khả năng thu thập dữ liệu từ cảm biến DHT22, gửi thông tin lên giao diện giám sát và hỗ trợ điều khiển các thiết bị như máy bơm, quạt và hệ thống phun sương.

Thông qua bài tập này, em có cơ hội vận dụng kiến thức lý thuyết vào thực hành, tìm hiểu sâu hơn về lập trình đa nhiệm, truyền thông IoT và xây dựng một hệ thống điều khiển tự động. Em xin chân thành cảm ơn Thầy Võ Quang Sơn đã truyền đạt kiến thức, hướng dẫn và tạo điều kiện để em hoàn thành bài tập. Đây là cơ hội quý báu giúp em củng cố kiến thức chuyên ngành và tích lũy thêm kinh nghiệm thực tế.

# CHƯƠNG 1. RTOS VÀ CÁC CƠ CHẾ GIAO TIẾP GIỮA CÁC TASK

## 1.1. Giới thiệu RTOS và ưu điểm

RTOS (Real-Time Operating System – hệ điều hành thời gian thực) là hệ điều hành được thiết kế để quản lý và thực thi các công việc có yêu cầu về thời gian đáp ứng xác định. Khác với hệ điều hành thông thường chủ yếu tập trung vào khả năng xử lý nhiều chương trình, RTOS chú trọng đến việc đảm bảo các tác vụ quan trọng được thực hiện đúng thời điểm và trong khoảng thời gian cho phép.

RTOS tổ chức chương trình thành các đơn vị công việc độc lập gọi là Task (tác vụ). Mỗi Task có thể được thiết lập mức độ ưu tiên, chu kỳ thực hiện và trạng thái hoạt động riêng. Bộ lập lịch của RTOS có nhiệm vụ phân chia thời gian xử lý cho các Task, lựa chọn tác vụ phù hợp để thực thi. Ngoài ra, RTOS còn cung cấp các cơ chế đồng bộ và trao đổi dữ liệu nhằm giúp các tác vụ phối hợp với nhau một cách hiệu quả.

Một số ưu điểm của RTOS:

- Đáp ứng thời gian tốt hơn: Các tác vụ được lập lịch dựa trên mức độ ưu tiên, giúp những công việc quan trọng được xử lý kịp thời.
- Hỗ trợ đa nhiệm: Cho phép nhiều tác vụ như đọc dữ liệu cảm biến, truyền dữ liệu qua mạng và điều khiển thiết bị hoạt động độc lập, đồng thời.
- Quản lý tài nguyên dùng chung: Cung cấp các cơ chế như Queue, Semaphore và Mutex để trao đổi dữ liệu, đồng bộ và hạn chế xung đột khi nhiều Task cùng truy cập tài nguyên.
- Dễ mở rộng và bảo trì: Chương trình được chia thành các Task riêng biệt, giúp việc kiểm tra lỗi, thay đổi hoặc bổ sung chức năng thuận tiện hơn.
- Tận dụng phần cứng đa lõi: Một số vi điều khiển như ESP32 hỗ trợ nhiều lõi xử lý, cho phép phân bố các Task lên Core 0 hoặc Core 1 để tổ chức hoạt động hợp lý hơn.

Nhờ những đặc điểm trên, RTOS được sử dụng rộng rãi trong các hệ thống nhúng, IoT, điều khiển tự động và giám sát thiết bị, nơi nhiều công việc cần được thực hiện đồng thời và có yêu cầu về thời gian đáp ứng.

## 1.2. FreeRTOS: ưu điểm và hạn chế

FreeRTOS là một hệ điều hành thời gian thực mã nguồn mở, có kích thước nhỏ gọn và được thiết kế chuyên biệt cho các hệ thống nhúng sử dụng vi điều khiển. FreeRTOS cung cấp các chức năng cơ bản như tạo và quản lý Task, lập lịch thực thi, trao đổi dữ liệu và đồng bộ giữa các tác vụ.

Đối với ESP32, FreeRTOS đã được tích hợp sẵn trong môi trường phát triển Arduino-ESP32 và ESP-IDF. Vì vậy, người lập trình có thể sử dụng các hàm API của FreeRTOS để tổ chức chương trình thành nhiều Task, thiết lập độ ưu tiên, lựa chọn lõi xử lý và quản lý tài nguyên mà không cần cài đặt một hệ điều hành riêng.

Ưu điểm:

- Nhẹ và tiết kiệm tài nguyên: Có kích thước nhỏ, phù hợp với các thiết bị nhúng có dung lượng RAM và bộ nhớ giới hạn.
- Hỗ trợ lập lịch ưu tiên: Cho phép thiết lập mức độ ưu tiên cho từng Task, giúp các tác vụ quan trọng được thực hiện trước khi cần thiết.
- Quản lý đa nhiệm hiệu quả: Hỗ trợ tạo, tạm dừng, tiếp tục và kết thúc Task, đồng thời cung cấp cơ chế trì hoãn để tránh việc tác vụ liên tục chiếm dụng CPU.
- Đa dạng cơ chế đồng bộ: Cung cấp Queue để trao đổi dữ liệu, Semaphore để đồng bộ sự kiện và Mutex để bảo vệ tài nguyên dùng chung.
- Phù hợp với ESP32: Hỗ trợ xử lý đa lõi, giúp phân chia công việc giữa Core 0 và Core 1, từ đó tổ chức hệ thống rõ ràng và hiệu quả hơn.

Hạn chế:

- Yêu cầu kiến thức lập trình cao hơn: Người lập trình cần hiểu cách hoạt động của Task, bộ lập lịch, độ ưu tiên và cơ chế đồng bộ.
- Có nguy cơ xảy ra lỗi đồng bộ: Nếu sử dụng Mutex, Semaphore hoặc Queue không đúng cách, hệ thống có thể gặp deadlock, mất dữ liệu hoặc tranh chấp tài nguyên.
- Tiêu tốn thêm bộ nhớ: Mỗi Task cần một vùng RAM riêng để lưu trữ Stack và thông tin quản lý. Việc tạo quá nhiều Task có thể làm tăng mức sử dụng bộ nhớ.
- Cần thiết kế thời gian hợp lý: FreeRTOS không tự động đảm bảo mọi tác vụ đáp ứng đúng thời hạn. Hiệu quả hoạt động còn phụ thuộc vào cách thiết lập chu kỳ, độ ưu tiên và phương pháp xử lý của người lập trình.

## 1.3. Cơ chế giao tiếp giữa các Task

### 1.3.1. Queue

Queue là hàng đợi dữ liệu theo nguyên tắc FIFO (vào trước – ra trước), cho phép Task gửi dữ liệu và Task khác nhận dữ liệu mà không cần truy cập trực tiếp biến nội bộ của nhau. Queue có thể chứa dữ liệu có cấu trúc và hỗ trợ chờ dữ liệu.

Queue (hàng đợi) là cơ chế trao đổi dữ liệu giữa các Task trong FreeRTOS, hoạt động theo nguyên tắc FIFO (First In – First Out), nghĩa là dữ liệu được đưa vào trước sẽ được lấy ra trước. Queue cho phép một Task gửi dữ liệu và Task khác nhận dữ liệu mà không cần truy cập trực tiếp vào các biến nội bộ của nhau. Ngoài ra, Queue có thể lưu trữ nhiều kiểu dữ liệu khác nhau, bao gồm cả các cấu trúc dữ liệu (Struct), và hỗ trợ cơ chế chờ khi hàng đợi chưa có dữ liệu hoặc đã đầy.

**Ưu điểm của Queue:**

- Trao đổi dữ liệu an toàn: Giúp các Task truyền và nhận dữ liệu thông qua một cơ chế trung gian, hạn chế việc truy cập trực tiếp vào dữ liệu của nhau.
- Hỗ trợ truyền dữ liệu có cấu trúc: Có thể lưu trữ nhiều thông tin trong cùng một phần tử, thuận tiện cho việc truyền dữ liệu cảm biến.
- Hỗ trợ đồng bộ giữa các Task: Task nhận có thể chờ cho đến khi có dữ liệu mới, tránh phải liên tục kiểm tra trạng thái và gây lãng phí tài nguyên CPU.
- Quản lý dữ liệu theo thứ tự: Dữ liệu được xử lý theo thứ tự gửi vào, phù hợp với các ứng dụng cần duy trì trình tự của thông tin.

**Ứng dụng trong bài thực hành:**

Trong chương trình, hệ thống sử dụng hai Queue với mục đích khác nhau:

- dataQueue: Lưu trữ dữ liệu kiểu SensorData, bao gồm nhiệt độ, độ ẩm và thời điểm lấy mẫu. Task_ReadData gửi dữ liệu vào Queue sau mỗi lần đọc cảm biến DHT22. Task_Transmit nhận dữ liệu từ Queue, đóng gói thành JSON và gửi lên MQTT Broker.
- dataAlert: Có độ dài một phần tử, dùng để truyền giá trị độ ẩm mới nhất từ Task_ReadData sang Task_Alert. Chương trình sử dụng xQueueOverwrite() để thay thế dữ liệu cũ bằng dữ liệu mới, giúp Task_Alert luôn xử lý giá trị độ ẩm cập nhật nhất.

Như vậy, Queue đóng vai trò trung gian giúp tách biệt quá trình thu thập dữ liệu, truyền dữ liệu và điều khiển hệ thống, giúp chương trình hoạt động ổn định và dễ quản lý hơn.

### 1.3.2. Semaphore và Mutex

Semaphore và Mutex là hai cơ chế đồng bộ được FreeRTOS cung cấp nhằm quản lý hoạt động giữa các Task và hạn chế xung đột khi sử dụng tài nguyên chung.

Semaphore là cơ chế đồng bộ thường được sử dụng để báo hiệu giữa các Task hoặc quản lý số lượng tài nguyên có thể truy cập. Ví dụ, một Task có thể gửi tín hiệu thông báo rằng dữ liệu đã sẵn sàng để Task khác tiếp tục xử lý. Semaphore thường được chia thành Binary Semaphore (Semaphore nhị phân) và Counting Semaphore (Semaphore đếm).

Mutex (Mutual Exclusion) là một dạng khóa dùng để bảo vệ tài nguyên chung, bảo đảm tại một thời điểm chỉ có một Task được phép truy cập vào vùng tài nguyên đang được bảo vệ. Khi một Task chiếm Mutex, các Task khác muốn sử dụng cùng tài nguyên phải chờ cho đến khi Mutex được giải phóng.

**Ưu điểm của Semaphore và Mutex:**

- Đồng bộ hoạt động giữa các Task: Giúp các tác vụ phối hợp với nhau theo đúng trình tự và thời điểm cần thiết.
- Bảo vệ tài nguyên dùng chung: Ngăn nhiều Task cùng truy cập và thay đổi dữ liệu trong một thời điểm, hạn chế sai lệch thông tin.
- Hạn chế xung đột dữ liệu: Đặc biệt hữu ích khi nhiều Task cùng sử dụng các tài nguyên như bộ nhớ, cổng giao tiếp hoặc thiết bị ngoại vi.
- Tăng tính ổn định của hệ thống: Giúp chương trình hoạt động có kiểm soát, tránh các lỗi phát sinh do truy cập đồng thời.

**Ứng dụng trong bài thực hành:**

Chương trình sử dụng hai Mutex để bảo vệ các tài nguyên dùng chung:

- serialMutex: Bảo vệ thao tác ghi dữ liệu ra Serial Monitor. Khi một Task đang thực hiện lệnh ghi Serial, các Task khác phải chờ, tránh tình trạng nhiều thông báo xuất hiện xen lẫn và gây khó khăn khi theo dõi kết quả.

- actuatorMutex: Bảo vệ các biến trạng thái của thiết bị gồm pumpState, fanState và mistPercent. Khi một Task cập nhật hoặc đọc các trạng thái này, Mutex giúp tránh việc các Task khác đồng thời thay đổi dữ liệu, gây sai lệch trạng thái điều khiển.

Nhìn chung, Semaphore và Mutex giúp các Task phối hợp và sử dụng tài nguyên chung an toàn hơn. Trong bài thực hành, Mutex được sử dụng trực tiếp để bảo vệ Serial và trạng thái thiết bị, góp phần nâng cao độ ổn định của hệ thống giám sát vườn cây.

# CHƯƠNG 2. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

## 2.1. Mô hình tổng thể

Hệ thống gồm ESP32, cảm biến DHT22, máy bơm, quạt và máy phun sương (có thể mô phỏng bằng LED/tải đầu ra trên Wokwi). ESP32 đọc dữ liệu môi trường, xử lý bằng các Task FreeRTOS và gửi dữ liệu qua Wi-Fi đến Broker EMQX bằng MQTT. Website kết nối cùng Broker để nhận dữ liệu, hiển thị và gửi lệnh điều khiển máy bơm/quạt.

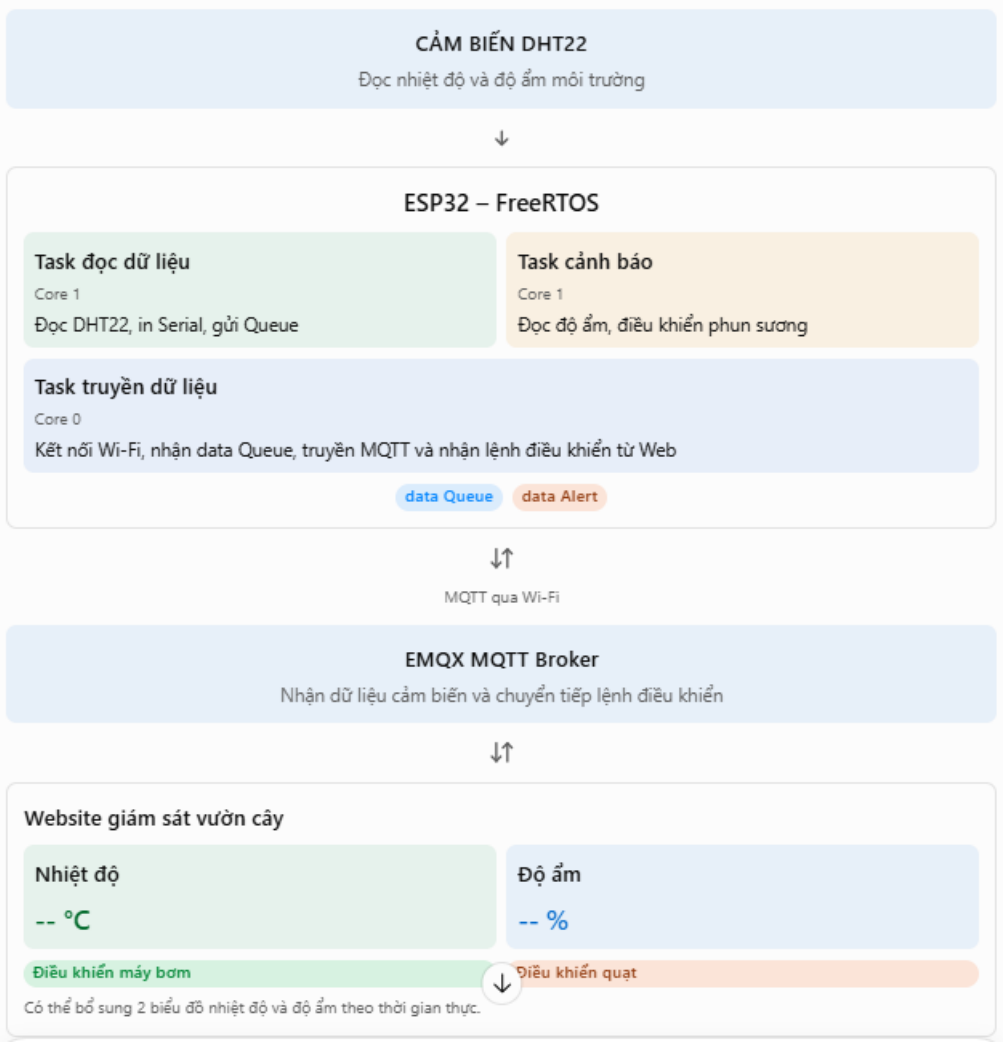



*Figure 1: Sơ đồ tổng thể hệ thống.*

## 2.2. Linh kiện và sơ đồ kết nối

Mô hình được triển khai trên Wokwi với ESP32 và DHT22. Các chân đầu ra mô phỏng trạng thái của máy bơm, quạt và máy phun sương.

| Thiết bị | Chân ESP32 | Chức năng |
|---|---|---|
| DHT22 DATA | GPIO 4 | Đọc nhiệt độ và độ ẩm |
| Máy bơm (LED xanh lá mô phỏng) | GPIO 18 | Bật/tắt từ Website |
| Quạt (LED đỏ mô phỏng) | GPIO 19 | Bật/tắt từ Website |
| Máy phun sương (tải PWM LED xanh) | GPIO 23 | Điều khiển công suất tự động |

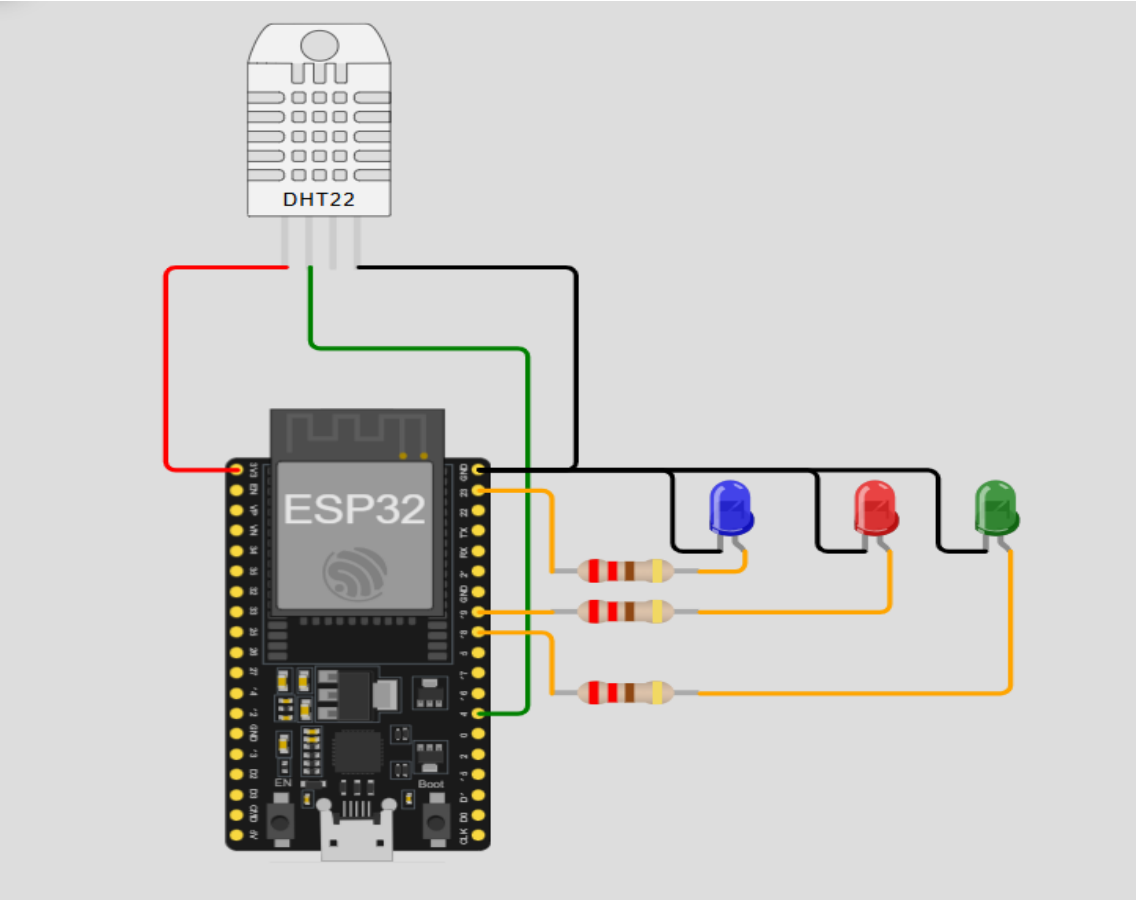



*Figure 2: Sơ đồ kết nối phần cứng trên Wokwi*

## 2.3. Nguyên lý hoạt động

1. Task_ReadData đọc DHT22 theo chu kỳ 2 giây, hiển thị Serial và đưa dữ liệu vào hai Queue.
2. Task_Transmit trên Core 0 kiểm tra Wi-Fi/MQTT, nhận dataQueue và gửi Telemetry lên Broker.
3. Task_Alert nhận độ ẩm từ dataAlert và điều khiển máy phun sương theo ngưỡng.
4. Website subscribe dữ liệu và trạng thái thiết bị; hai nút điều khiển publish lệnh bật/tắt máy bơm và quạt.
5. ESP32 nhận lệnh trong mqttCallback(), cập nhật GPIO và gửi trạng thái mới lên Broker.

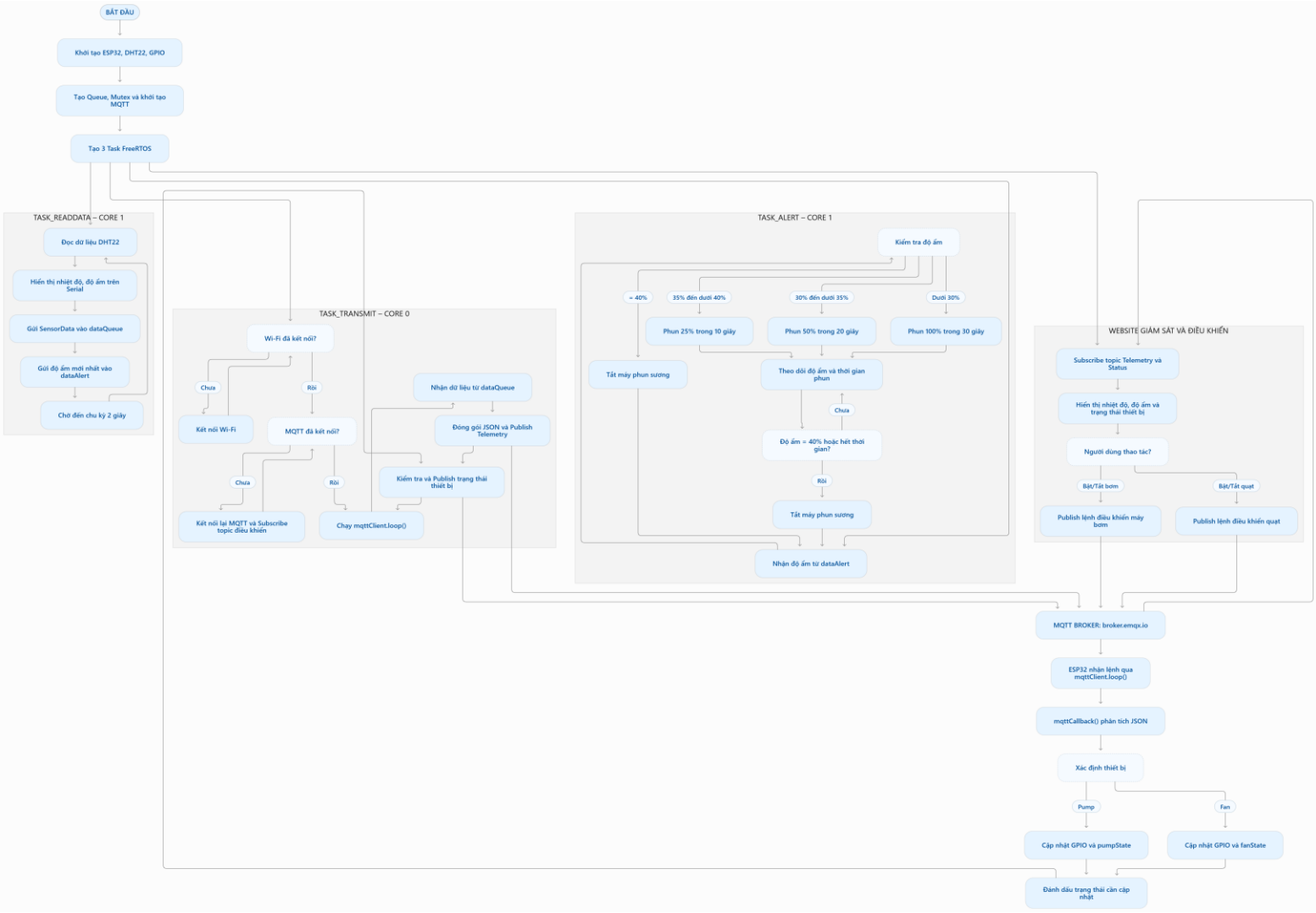



*Figure 3: Lưu đồ nguyên lý hoạt động tổng thể của hệ thống*

# CHƯƠNG 3. TRIỂN KHAI FREERTOS TRÊN ESP32

## 3.1. Bảng thông tin các Task

Chương trình tạo ba Task bằng xTaskCreatePinnedToCore(). Task đọc cảm biến và Task cảnh báo chạy trên Core 1; Task truyền dữ liệu chạy trên Core 0. Cả ba Task có độ ưu tiên 2.

| Tên Task | Core | Ưu tiên | Stack | Nhiệm vụ |
|---|---|---|---|---|
| Task_ReadData | 1 | 2 | 4096 | Đọc DHT22 mỗi 2 giây, in Serial, gửi SensorData vào dataQueue và độ ẩm vào dataAlert. |
| Task_Alert | 1 | 2 | 4096 | Nhận độ ẩm mới nhất, xác định mức phun và điều khiển PWM GPIO23. |
| Task_Transmit | 0 | 2 | 6144 | Kết nối Wi-Fi/MQTT, xử lý bản tin, nhận dataQueue, gửi Telemetry và trạng thái thiết bị. |

Lưu ý: đơn vị stack và giới hạn thực tế phụ thuộc phiên bản Arduino-ESP32/FreeRTOS được sử dụng; khi chuyển nền tảng cần kiểm tra lại yêu cầu stack.

## 3.2. Khởi tạo Queue và Mutex

```
dataQueue = xQueueCreate(10, sizeof(SensorData));
dataAlert = xQueueCreate(1, sizeof(float));
serialMutex = xSemaphoreCreateMutex();
actuatorMutex = xSemaphoreCreateMutex();
```

dataQueue chứa tối đa 10 mẫu SensorData. Nếu Queue đầy, Task_ReadData loại bỏ mẫu cũ nhất rồi thêm mẫu mới. dataAlert chỉ giữ độ ẩm mới nhất, phù hợp với điều khiển theo trạng thái hiện tại. Hai Mutex bảo vệ Serial và biến trạng thái cơ cấu chấp hành. Chương trình kiểm tra kết quả khởi tạo; nếu tài nguyên không được tạo thành công thì dừng để tránh chạy thiếu tài nguyên.

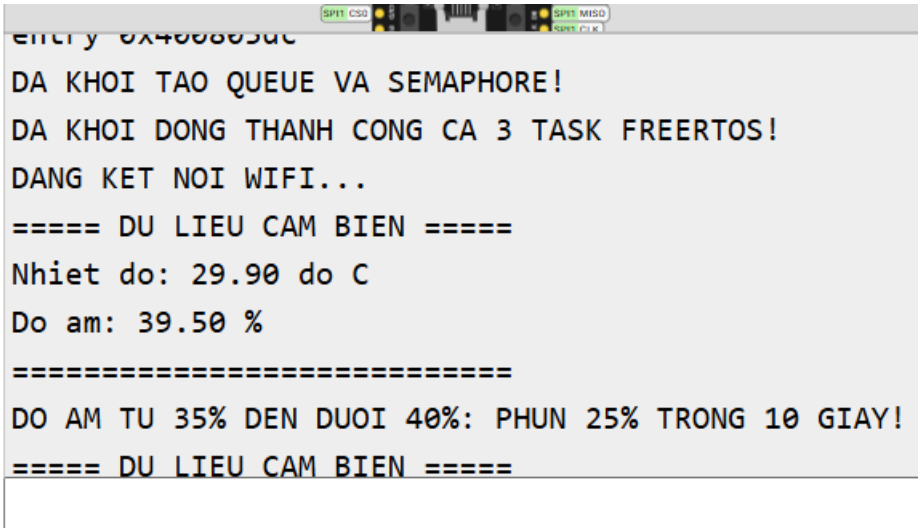



*Figure 4: Khởi tạo Queue, Semaphore và các Task*

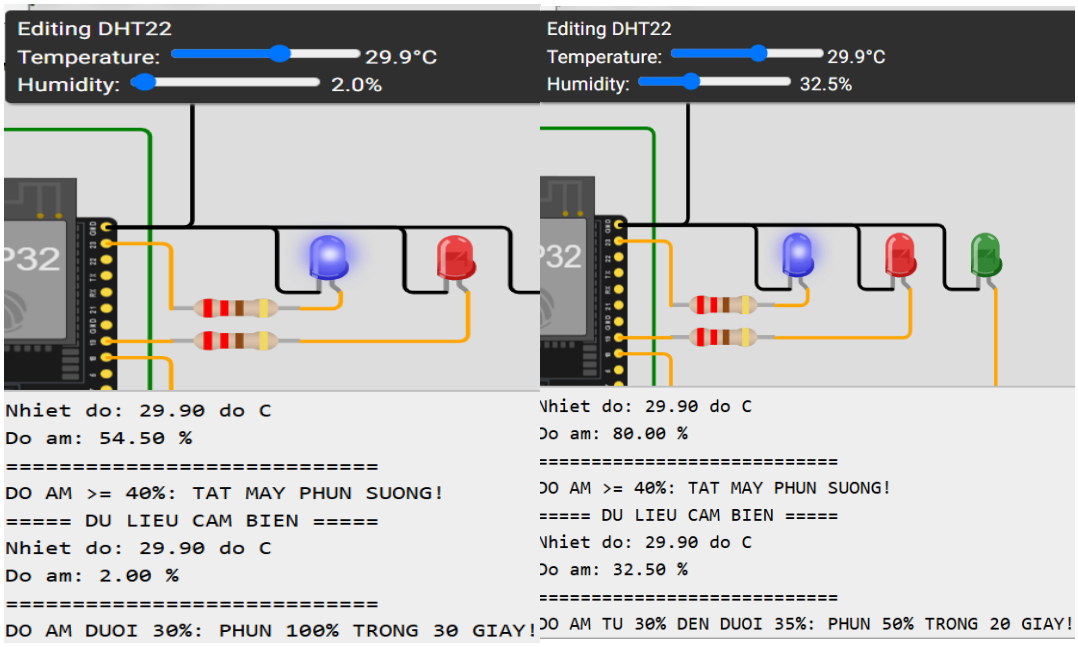
### 3.3. Thuật toán điều khiển máy phun sương

Hệ thống hướng đến vùng độ ẩm mục tiêu 40%–60%. Theo code hiện tại, khi độ ẩm dưới 40%, hàm getMistSetting() chọn công suất và thời gian phun như sau:

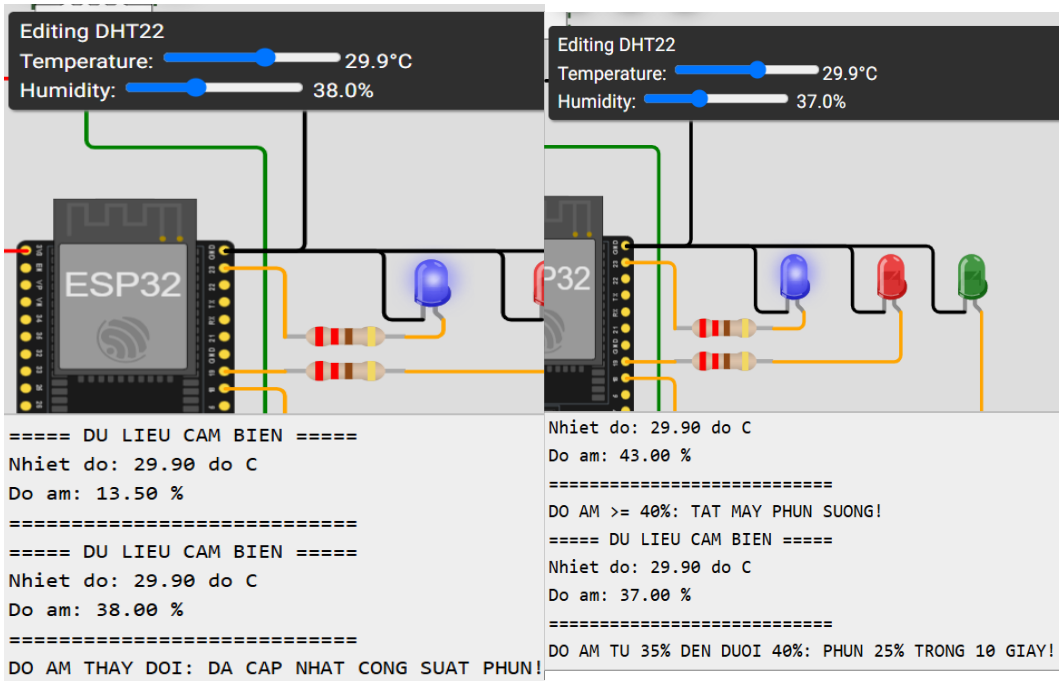
| Độ ẩm | Công suất | Thời gian |
|---|---|---|
| Từ 40% trở lên | 0% – Tắt | Không phun |
| Từ 35% đến dưới 40% | 25% | 10 giây |
| Từ 30% đến dưới 35% | 50% | 20 giây |
| Dưới 30% | 100% | 30 giây |

Hàm setMistPercent() chuyển phần trăm công suất thành giá trị PWM từ 0 đến 255 theo công thức duty = percent × 255 / 100, sau đó xuất tín hiệu tại GPIO23. Trong lúc phun, Task_Alert tiếp tục kiểm tra độ ẩm mới. Nếu độ ẩm đạt từ 40% trở lên, máy tắt; nếu chuyển sang khoảng khác, công suất và thời gian được cập nhật.

Lưu ý: code hiện tại dùng 40% làm ngưỡng dừng phun. Vùng 40%–60% là vùng mục tiêu được nêu trong yêu cầu;




a) b)




c) d)

*Figure 5: a) Dưới 30% độ ẩm ( máy phun chạy 100% trong 30s, LED sáng mạnh nhất)*

*b) Độ ẩm 30-35% ( máy phun chạy 50% công suất trong 20s, LED sáng vừa)*
*c) ) Độ ẩm 35-40% ( máy phun chạy 25% công suất trong 10s, LED sáng yếu nhất)*

*d) Độ ẩm >40% (máy phun sương tắt)*

# CHƯƠNG 4. GIAO TIẾP MQTT VÀ THIẾT KẾ WEBSITE

## 4.1. Kết nối Wi-Fi và MQTT

ESP32 sử dụng Wi-Fi Wokwi-GUEST. Broker cấu hình là broker.emqx.io, cổng TCP 1883. Thư viện PubSubClient đảm nhiệm kết nối, đăng ký Topic, nhận và gửi bản tin. Theo mô hình publish/subscribe, ESP32 publish Telemetry và trạng thái; Website subscribe dữ liệu và publish lệnh điều khiển.

Client ID: esp32_lab3_cafe_4729. Các Topic phải được cấu hình thống nhất ở ESP32 và Website:

| Topic | Chiều truyền | Dữ liệu |
|---|---|---|
| lab3/garden/cafe_4729/telemetry | ESP32 → Broker → Website | temperature, humidity, sampleTime |
| lab3/garden/cafe_4729/control/pump/set | Website → Broker → ESP32 | {"state": true/false} |
| lab3/garden/cafe_4729/control/fan/set | Website → Broker → ESP32 | {"state": true/false} |
| lab3/garden/cafe_4729/status/actuators | ESP32 → Broker → Website | pump, fan, mistPercent |

Khi MQTT kết nối thành công, ESP32 đăng ký hai Topic điều khiển. Task_Transmit gọi mqttClient.loop() thường xuyên để xử lý bản tin đến. Nếu mất kết nối, reconnectMQTT() thử kết nối lại với khoảng cách tối thiểu 5 giây.

## 4.2. Cấu trúc bản tin JSON

Telemetry:

```
{"temperature": 27.4, "humidity": 38.0, "sampleTime": 123456}
```

Lệnh điều khiển máy bơm hoặc quạt:

```
{"state": true}
```

Trạng thái cơ cấu chấp hành:

```
{"pump": true, "fan": false, "mistPercent": 25}
```

Website cần sử dụng chính xác các Topic và tên trường JSON trên. Nếu thay đổi một phía mà không cập nhật phía còn lại, dữ liệu hoặc lệnh có thể không được trao đổi.

## 4.3. Thiết kế Website

Website “Vườn cây công nghiệp thông minh” nhận dữ liệu từ MQTT Broker và cung cấp giao diện giám sát, điều khiển. Các thành phần gồm:

- Hai thẻ số hiển thị nhiệt độ (°C) và độ ẩm (%), cập nhật khi nhận Telemetry.
- Hai biểu đồ đường thể hiện biến thiên nhiệt độ và độ ẩm theo thời gian.

- Hai nút bật/tắt máy bơm và quạt; Website publish JSON đến Topic điều khiển tương ứng.
- Khu vực máy phun sương hiển thị độ ẩm mới nhất, công suất và mô tả ngưỡng. Ví dụ 38%: độ ẩm từ 35% đến dưới 40% thì bật 25% công suất trong 10 giây.
- Khu vực hiển thị trạng thái kết nối MQTT và trạng thái cơ cấu chấp hành.

Máy bơm và quạt được điều khiển thủ công từ Website. Máy phun sương không có nút điều khiển thủ công trong code hiện tại mà do Task_Alert điều khiển tự động.

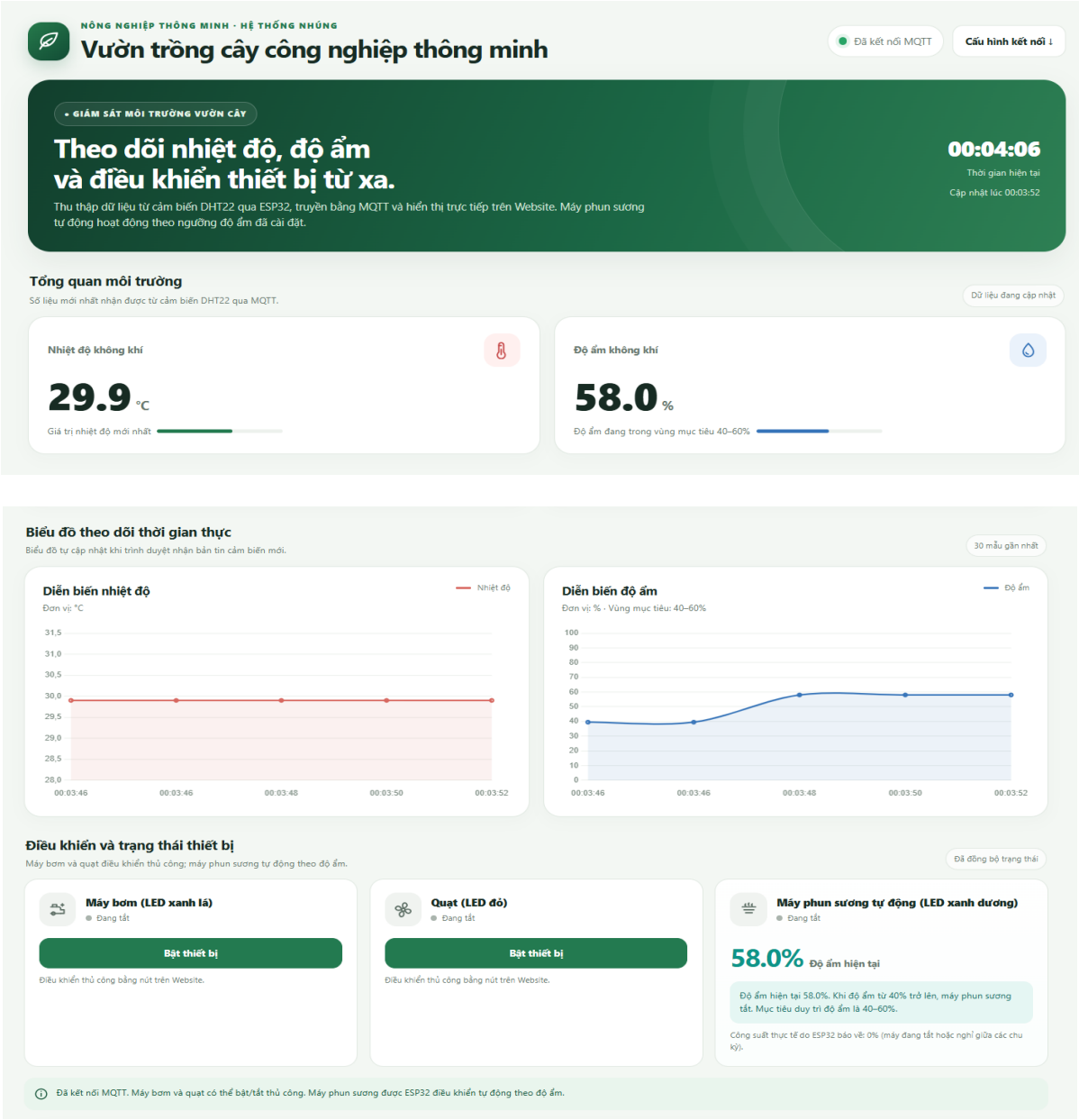



*Figure 6: Giao diện chính của Website giám sát vườn cây công nghiệp*

# CHƯƠNG 5. KẾT QUẢ THỰC HIỆN VÀ KIỂM THỬ

Các kết quả dưới đây cần được xác nhận bằng ảnh chụp từ lần chạy mô phỏng thực tế. Không nên đánh dấu đạt nếu chưa quan sát được kết quả tương ứng.

## 5.1. Kết quả giám sát và truyền dữ liệu

### a) Kết quả mô phỏng mạch Wokwi và hiển thị dữ liệu trên Serial Monitor

Sau khi hoàn thành kết nối các linh kiện và nạp chương trình cho ESP32 trên Wokwi, hệ thống đã có thể đọc dữ liệu nhiệt độ và độ ẩm từ cảm biến DHT22 theo chu kỳ 2 giây. Các giá trị thu thập được hiển thị trên Serial Monitor, đồng thời được đóng gói thành bản tin JSON để phục vụ quá trình truyền dữ liệu qua MQTT.

Kết quả cho thấy ESP32 có thể thu thập dữ liệu cảm biến và thực hiện quá trình truyền Telemetry theo đúng thiết kế.

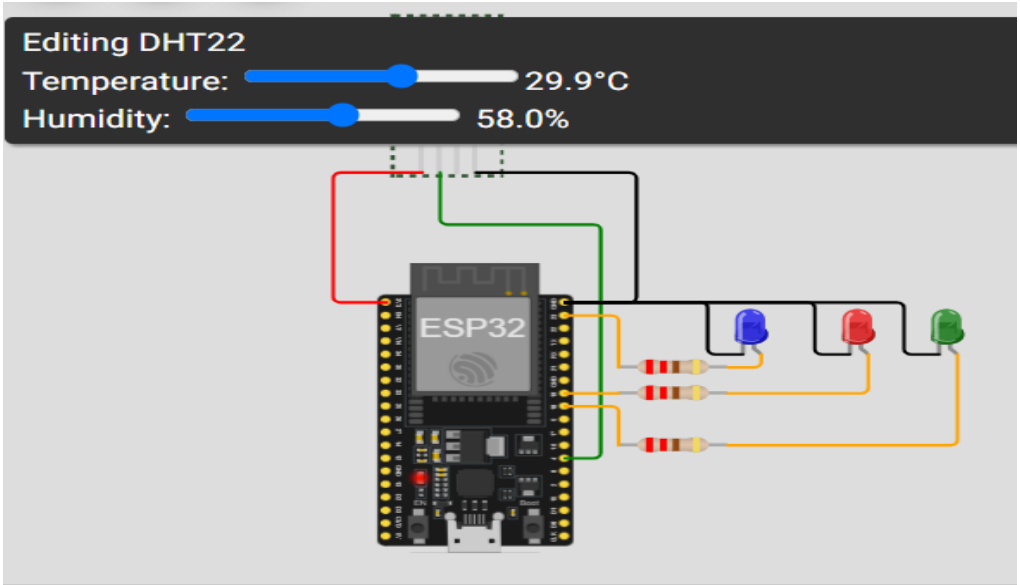



*Figure 7: Sơ đồ mạch, Serial Monitor hiển thị nhiệt độ, độ ẩm và dữ liệu Telemetry*

### b) Kết quả hiển thị dữ liệu trên Website

Website được kết nối với MQTT Broker để nhận dữ liệu Telemetry do ESP32 gửi lên. Các thông tin như nhiệt độ và độ ẩm được cập nhật trên giao diện, giúp người dùng theo dõi tình trạng môi trường của vườn cây.

Khi giá trị cảm biến trên Wokwi thay đổi, dữ liệu được ESP32 gửi lên Broker và Website nhận được giá trị mới tương ứng. Qua đó, chức năng giám sát dữ liệu từ xa đã được thực hiện theo yêu cầu đề bài.

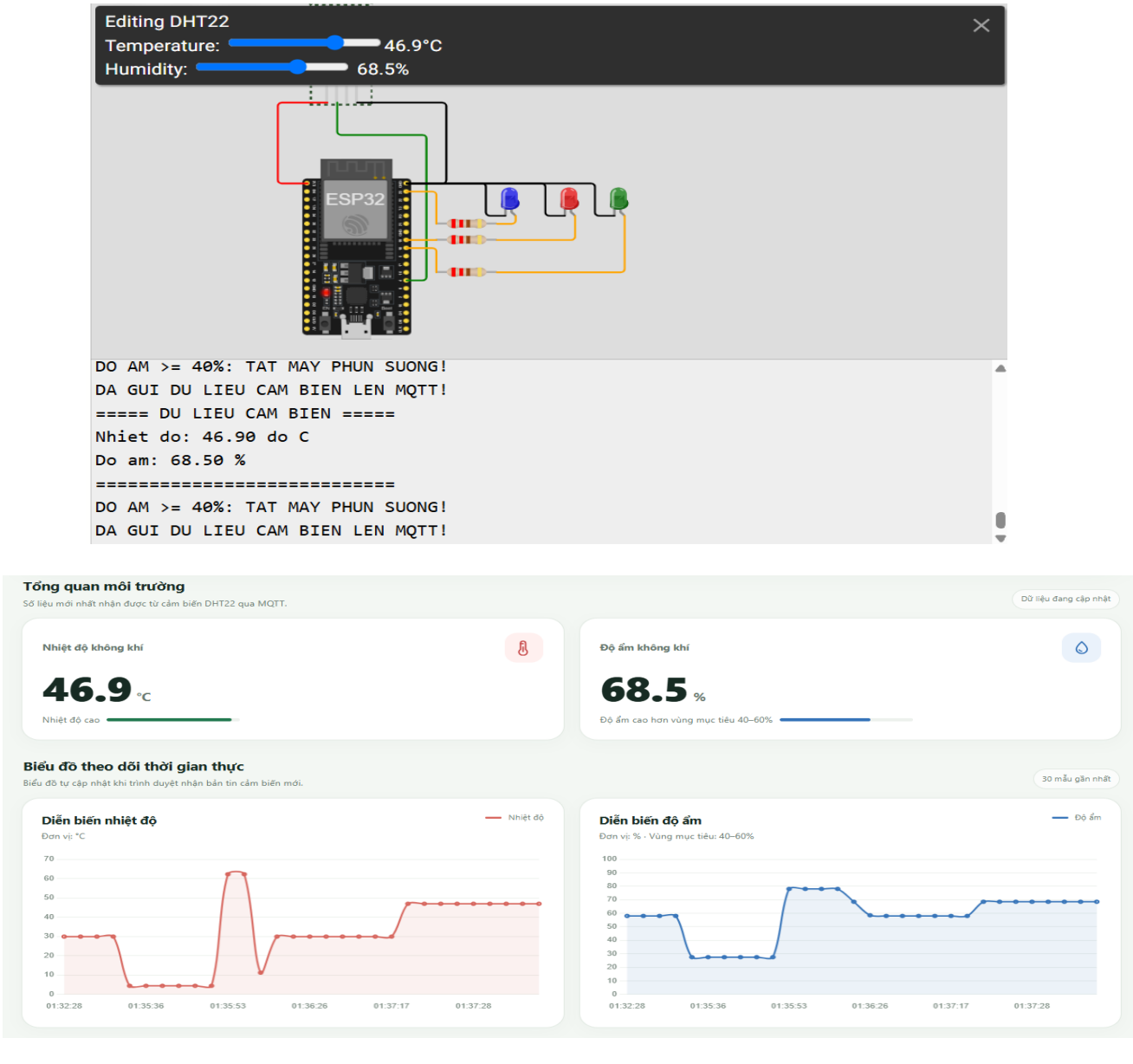


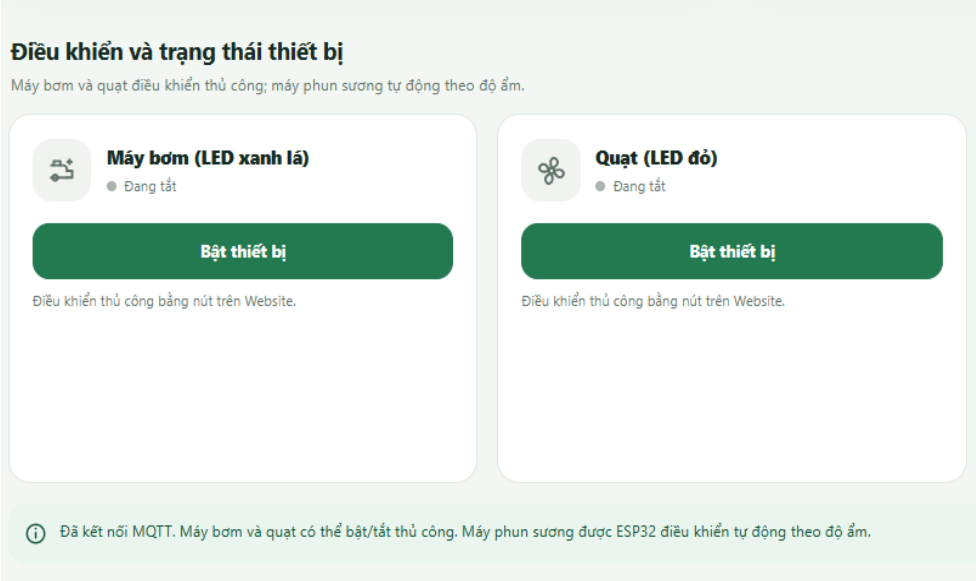

*Figure 7: Dữ liệu trên Website thay đổi tương ứng khi điều chỉnh giá trị cảm biến trên Wokwi*

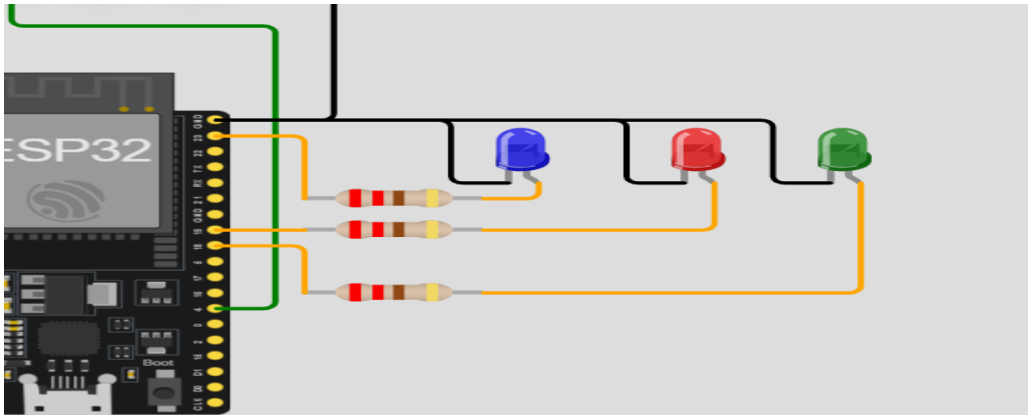
## 5.2. Kết quả điều khiển thiết bị

### a) Kết quả điều khiển máy bơm và quạt

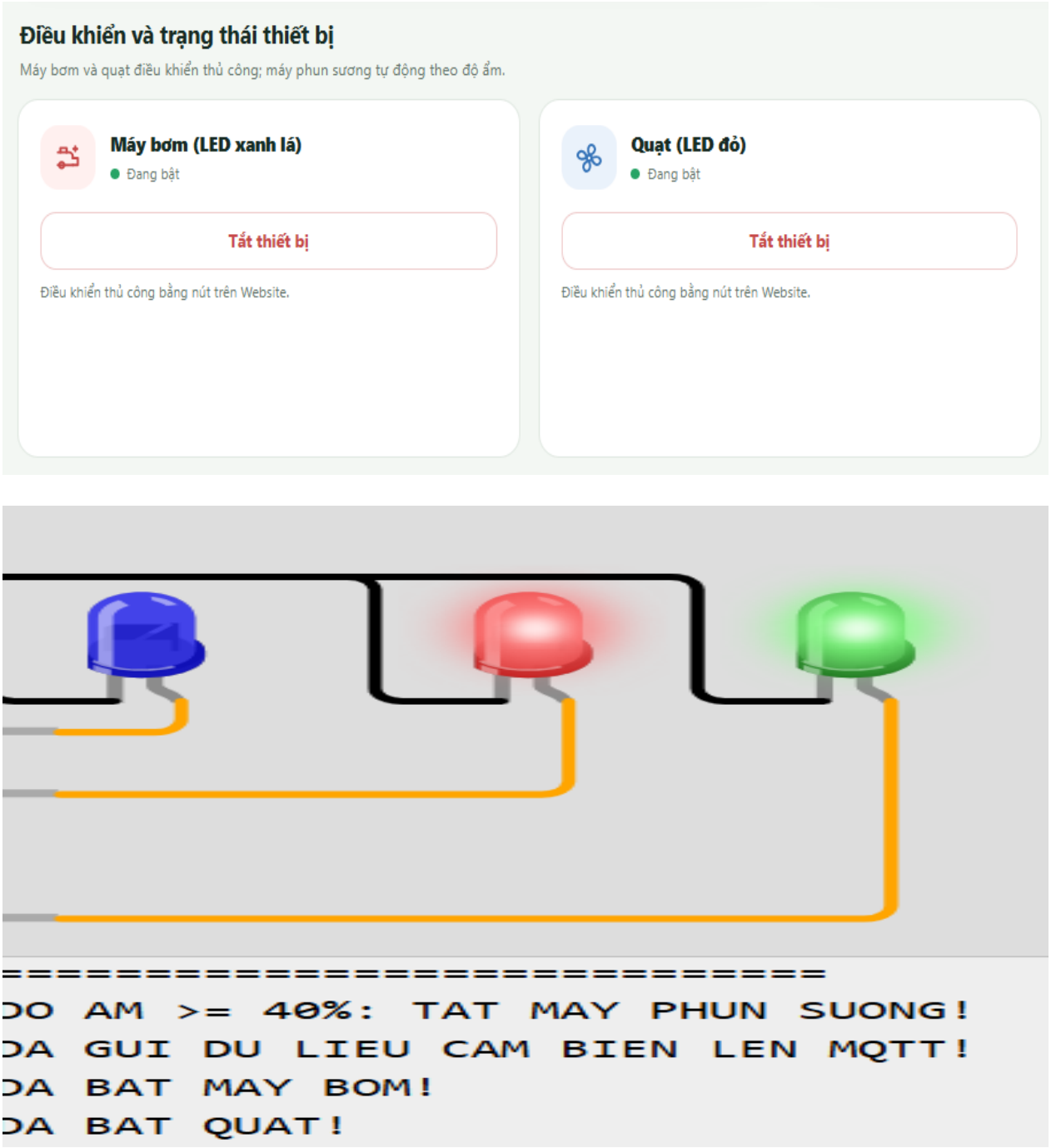
Website cung cấp hai nút điều khiển cho phép người dùng bật hoặc tắt máy bơm và quạt ở chế độ thủ công. Khi người dùng thao tác trên giao diện, Website gửi lệnh điều khiển đến MQTT Broker. ESP32 nhận bản tin thông qua hàm mqttCallback(), phân tích nội dung lệnh và cập nhật trạng thái GPIO tương ứng.

Kết quả kiểm thử cho thấy các thiết bị có thể được bật hoặc tắt thông qua Website. Trạng thái sau khi điều khiển cũng được ESP32 gửi ngược lên Broker để Website cập nhật và hiển thị, giúp người dùng dễ dàng theo dõi hoạt động của hệ thống.




*Hình 8: Kết quả máy bơm (LED xanh lá) và quạt (LED đỏ) ở trạng thái tắt*

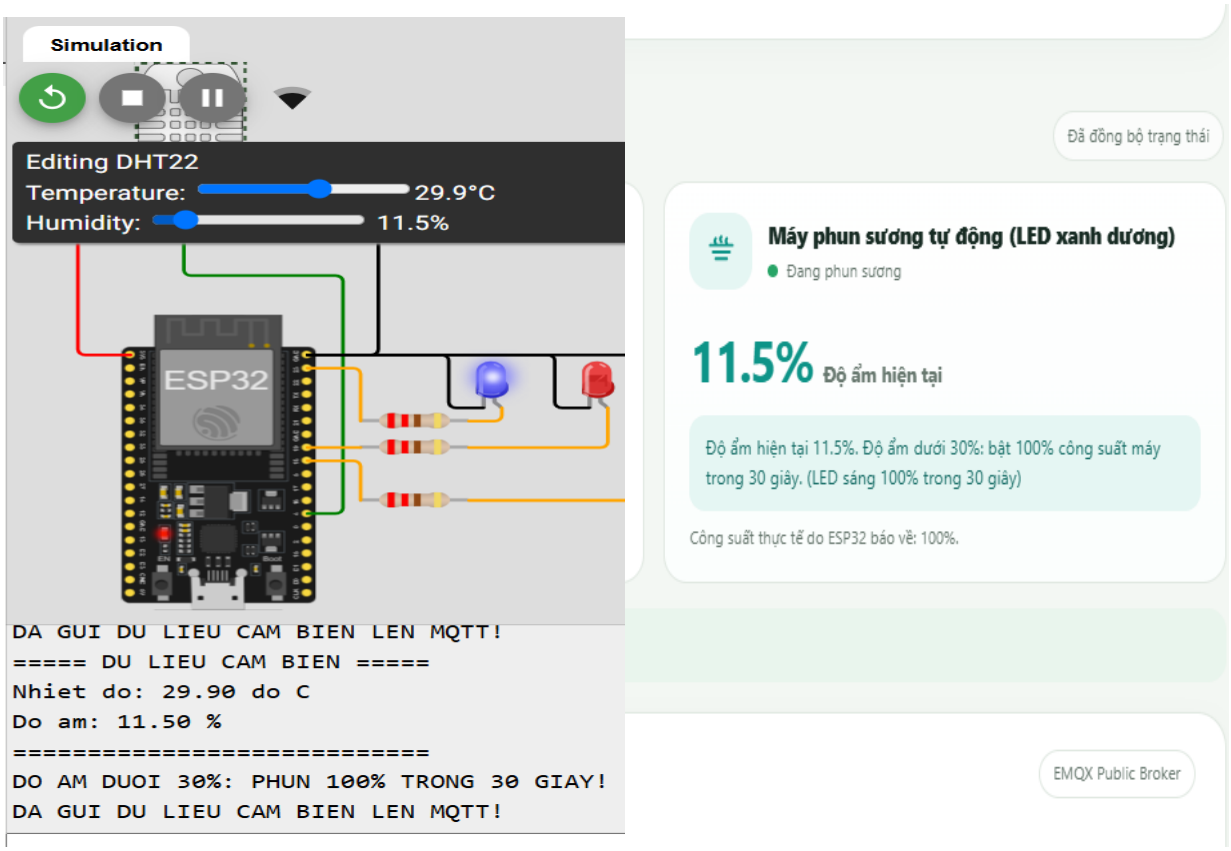



*Hình 9: Kết quả máy bơm (LED xanh lá) và quạt (LED đỏ) bật sau khi điều khiển trên Website*

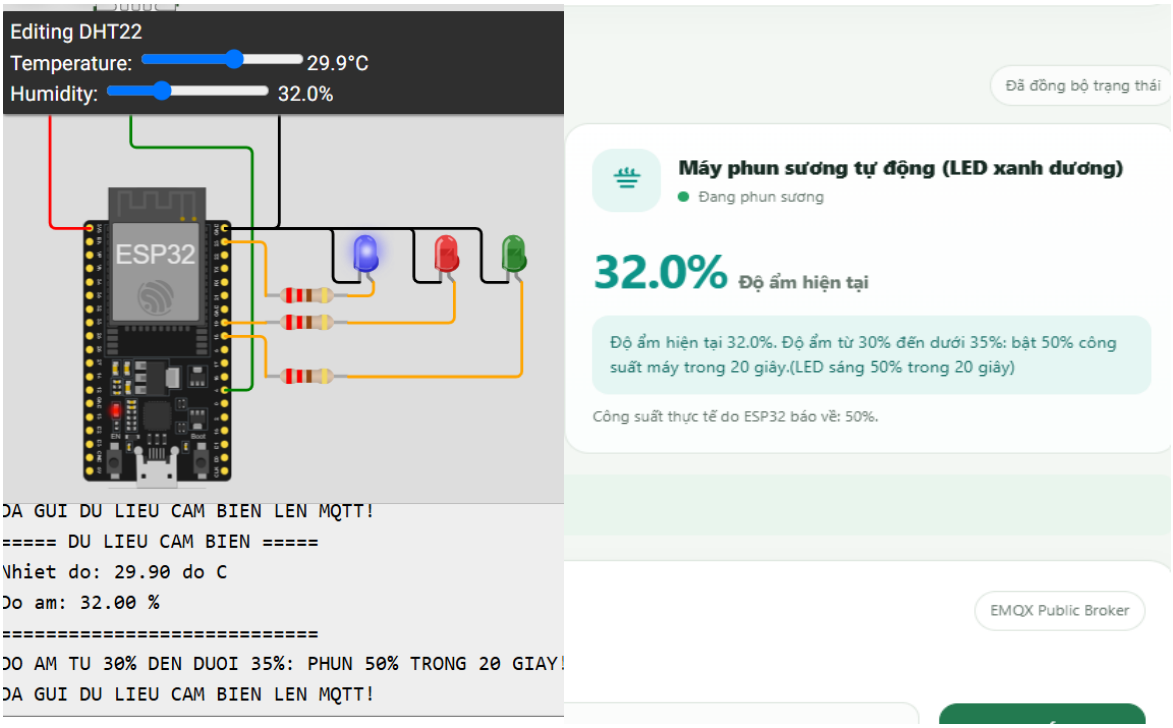
**b) Kết quả hoạt động tự động của máy phun sương**

Hệ thống được thiết lập với ba mức hoạt động: khi độ ẩm từ 35% đến dưới 40%, máy phun sương hoạt động ở mức 25% trong 10 giây; khi độ ẩm từ 30% đến dưới 35%, hoạt động ở mức 50% trong 20 giây; khi độ ẩm dưới 30%, hoạt động ở mức 100% trong 30 giây. Khi độ ẩm đạt từ 40% trở lên, hệ thống tắt máy phun sương.

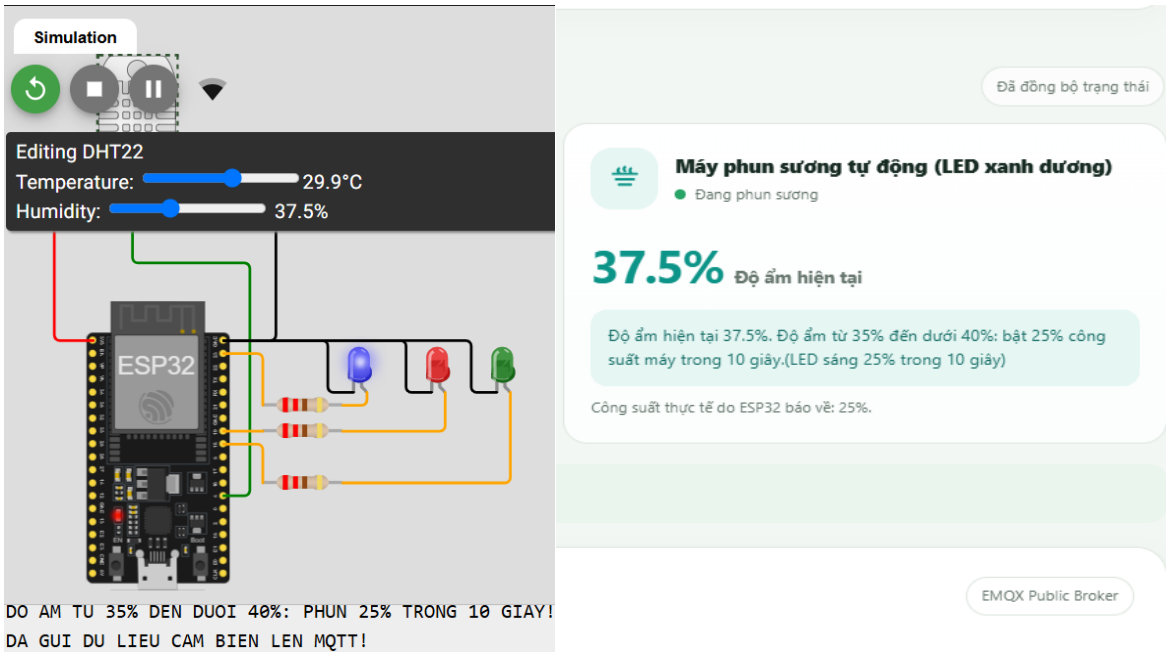
Qua kiểm thử, chức năng tự động có thể lựa chọn mức phun theo từng ngưỡng độ ẩm và tự động dừng khi đạt điều kiện đã thiết lập. Điều này giúp hệ thống chủ động điều chỉnh độ ẩm, hạn chế việc phải điều khiển máy phun sương bằng tay.



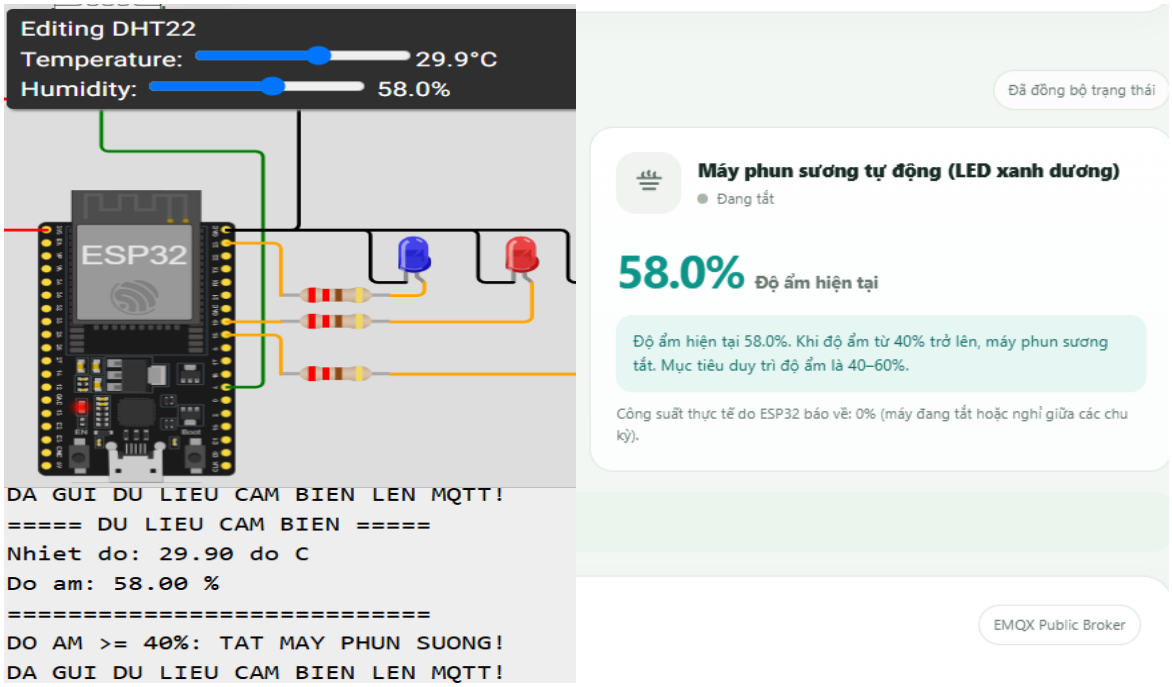

*Hình 10: Kết quả mô phỏng máy phun sương hoạt động ở mức 100% khi độ ẩm dưới 30%, và hiển thị chế độ hoạt động tự động lên Website*




*Hình 10: Kết quả mô phỏng máy phun sương hoạt động ở mức 100% khi độ ẩm 30-35%, và hiển thị chế độ hoạt động tự động lên Website*




*Hình 11: Kết quả mô phỏng máy phun sương hoạt động ở mức 100% khi độ ẩm dưới 35%-40%, và hiển thị chế độ hoạt động tự động lên Website*




*Hình 12: Kết quả mô phỏng máy phun sương hoạt động ở mức 100% khi độ ẩm >40%, và hiển thị chế độ hoạt động tự động lên Website*

# CHƯƠNG 6. CODE VÀ PHÂN TÍCH CODE

## 6.1. Link code

Link code trên Wokwi hoặc dưới link sau đây:
https://drive.google.com/drive/folders/1SYdPJ18-YBDyV55R6eJEW_ZYFvUTiCvM?usp=drive_link

## 6.2. Phân tích code

### a) Khai báo thư viện và cấu hình phần cứng

Đầu tiên, chương trình khai báo các thư viện cần thiết để ESP32 thực hiện việc kết nối mạng, giao tiếp MQTT, xử lý dữ liệu JSON, đọc cảm biến DHT22 và quản lý các tác vụ bằng FreeRTOS.

Các thư viện có chức năng chính như sau:

- WiFi.h: Hỗ trợ ESP32 kết nối mạng Wi-Fi.
- PubSubClient.h: Cung cấp các hàm giao tiếp MQTT như kết nối Broker, Publish và Subscribe.
- ArduinoJson.h: Hỗ trợ tạo và phân tích các bản tin JSON.
- DHT.h: Đọc dữ liệu nhiệt độ và độ ẩm từ cảm biến DHT22.
- Các thư viện FreeRTOS: Hỗ trợ tạo Task, Queue và Mutex để quản lý hoạt động đa nhiệm.

Tiếp theo, chương trình khai báo các chân GPIO được sử dụng:

```
#define DHT_PIN   4

#define DHT_TYPE  DHT22

#define PUMP_PIN  18

#define FAN_PIN   19

#define MIST_PIN  23

DHT dht(DHT_PIN, DHT_TYPE);
```

Trong đó, cảm biến DHT22 được kết nối với GPIO4; máy bơm sử dụng GPIO18; quạt sử dụng GPIO19 và máy phun sương sử dụng GPIO23. Chân GPIO23 được điều khiển bằng PWM để thay đổi mức công suất phun sương.

**b) Cấu hình Wi-Fi và MQTT Broker**

ESP32 sử dụng mạng Wi-Fi của Wokwi để truy cập Internet và kết nối với MQTT Broker công cộng của EMQX.

```
const char* WIFI_SSID = "Wokwi-GUEST";

const char* WIFI_PASSWORD = "";

const char* MQTT_HOST = "broker.emqx.io";

const int MQTT_PORT = 1883;

const char* MQTT_CLIENT_ID = "esp32_lab3_cafe_4729";
```

Trong đó:

- WIFI_SSID và WIFI_PASSWORD: Thông tin mạng Wi-Fi mà ESP32 sử dụng.
- MQTT_HOST: Địa chỉ máy chủ MQTT Broker.
- MQTT_PORT: Cổng giao tiếp MQTT, sử dụng cổng 1883.
- MQTT_CLIENT_ID: Mã định danh của ESP32 khi kết nối đến Broker.

Chương trình thiết lập bốn MQTT Topic để phân chia các loại dữ liệu và lệnh điều khiển:

```
const char* TOPIC_TELEMETRY =
  "lab3/garden/cafe_4729/telemetry";

const char* TOPIC_PUMP =
  "lab3/garden/cafe_4729/control/pump/set";

const char* TOPIC_FAN =
  "lab3/garden/cafe_4729/control/fan/set";

const char* TOPIC_STATUS =
  "lab3/garden/cafe_4729/status/actuators";
```

Ý nghĩa của từng Topic:

| MQTT Topic | Chức năng |
|---|---|
| .../telemetry | ESP32 gửi dữ liệu nhiệt độ và độ ẩm lên Broker. |
| .../control/pump/set | Nhận lệnh bật/tắt máy bơm. |
| .../control/fan/set | Nhận lệnh bật/tắt quạt. |
| .../status/actuators | ESP32 gửi trạng thái hiện tại của các thiết bị. |

Việc phân chia Topic giúp dữ liệu giám sát, lệnh điều khiển và trạng thái thiết bị được quản lý riêng biệt, thuận tiện cho quá trình trao đổi thông tin giữa ESP32 và Website.

**c) Khởi tạo Queue, Mutex và các Task FreeRTOS**

Chương trình sử dụng cấu trúc SensorData để lưu trữ đầy đủ thông tin cảm biến:

```
struct SensorData {
  float temperature;
  float humidity;
  unsigned long sampleTime;
};
```

Cấu trúc này gồm nhiệt độ, độ ẩm và thời điểm lấy mẫu. Hai Queue được sử dụng để truyền dữ liệu giữa các Task:

```
dataQueue = xQueueCreate(10, sizeof(SensorData));
dataAlert = xQueueCreate(1, sizeof(float));
serialMutex = xSemaphoreCreateMutex();
actuatorMutex = xSemaphoreCreateMutex();
```

Trong đó, dataQueue có thể lưu tối đa 10 mẫu dữ liệu cảm biến; dataAlert chỉ lưu một giá trị độ ẩm mới nhất. Hai Mutex lần lượt bảo vệ thao tác ghi Serial và các biến trạng thái thiết bị.

Sau khi khởi tạo tài nguyên, chương trình tạo ba Task và phân bố chúng lên hai lõi xử lý của ESP32:

```
xTaskCreatePinnedToCore(
  Task_Alert, "Task_Alert",
  4096, NULL, 2, NULL, 1
);
xTaskCreatePinnedToCore(
  Task_ReadData, "Task_ReadData",
  4096, NULL, 2, NULL, 1
);
xTaskCreatePinnedToCore(
  Task_Transmit, "Task_Transmit",
  6144, NULL, 2, NULL, 0
);
```

Cụ thể:

- Task_ReadData: Chạy trên Core 1, thực hiện đọc dữ liệu cảm biến.
- Task_Alert: Chạy trên Core 1, xử lý độ ẩm và điều khiển máy phun sương.
- Task_Transmit: Chạy trên Core 0, quản lý kết nối Wi-Fi, MQTT và truyền dữ liệu.

Cả ba Task đều được đặt độ ưu tiên bằng 2. Việc phân chia này giúp các chức năng thu thập dữ liệu, điều khiển tự động và truyền thông mạng được tổ chức độc lập, đồng thời tận dụng khả năng xử lý đa lõi của ESP32.

## 6.2. Quá trình kết nối và cập nhật dữ liệu

### a) Quá trình kết nối Wi-Fi

Việc kết nối Wi-Fi được thực hiện thông qua hàm connectWiFi():

```
void connectWiFi() {
```

```
  if (WiFi.status() == WL_CONNECTED) {
    return;
  }
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

Đầu tiên, chương trình kiểm tra trạng thái Wi-Fi. Nếu ESP32 đã kết nối thành công thì không cần thực hiện lại. Ngược lại, ESP32 chuyển sang chế độ Station và bắt đầu kết nối với mạng đã khai báo.

Trong thời gian chờ kết nối, Task sử dụng vTaskDelay() để tạm dừng 500 ms giữa các lần kiểm tra. Khi Wi-Fi được kết nối, chương trình tiếp tục thực hiện các bước kết nối MQTT.

**b) Quá trình kết nối MQTT Broker**

Sau khi có kết nối Wi-Fi, ESP32 sử dụng hàm reconnectMQTT() để kết nối đến máy chủ MQTT:

```
void reconnectMQTT() {
  if (mqttClient.connected()) {
    return;
  }
  if (millis() - lastMqttAttempt < 5000 &&
      lastMqttAttempt != 0) {
```

```
    return;
  }
  lastMqttAttempt = millis();
  if (mqttClient.connect(MQTT_CLIENT_ID)) {
    mqttClient.subscribe(TOPIC_PUMP);
    mqttClient.subscribe(TOPIC_FAN);


    statusDirty = true;
    publishActuatorStatus();
  }
}
```

Chương trình kiểm tra trạng thái kết nối MQTT trước khi thực hiện kết nối mới. Nếu kết nối thất bại, ESP32 sẽ chờ ít nhất 5 giây trước lần thử tiếp theo, tránh việc liên tục gửi yêu cầu kết nối đến Broker.

Khi kết nối thành công, ESP32 đăng ký theo dõi hai Topic điều khiển máy bơm và quạt bằng hàm subscribe(). Nhờ đó, thiết bị có thể nhận lệnh từ Website thông qua MQTT Broker. Chương trình cũng đánh dấu trạng thái thiết bị cần được cập nhật và gửi trạng thái hiện tại lên Broker.

**c) Quá trình đọc dữ liệu cảm biến và đưa vào Queue**

Việc thu thập dữ liệu được thực hiện bởi Task_ReadData trên Core 1. Cảm biến DHT22 được đọc theo chu kỳ 2 giây:

```
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();
if (isnan(temperature) || isnan(humidity)) {
  logLine("LOI: KHONG DOC DUOC DU LIEU TU DHT22!");
} else {
```

```
  logSensor(temperature, humidity);
  SensorData data;
  data.temperature = temperature;
  data.humidity = humidity;
  data.sampleTime = millis();
  xQueueSend(dataQueue, &data, 0);
  xQueueOverwrite(dataAlert, &humidity);
}
vTaskDelayUntil(
  &lastWakeTime,
  pdMS_TO_TICKS(SENSOR_INTERVAL)
);
```

Sau khi đọc cảm biến, chương trình kiểm tra dữ liệu có hợp lệ hay không. Nếu dữ liệu hợp lệ, nhiệt độ và độ ẩm được hiển thị trên Serial Monitor thông qua hàm logSensor().

Tiếp theo, chương trình tạo một biến kiểu SensorData để lưu thông tin vừa thu thập. Dữ liệu được gửi vào hai Queue với mục đích khác nhau:

- dataQueue: Nhận đầy đủ nhiệt độ, độ ẩm và thời điểm lấy mẫu để phục vụ quá trình truyền Telemetry.
- dataAlert: Nhận riêng giá trị độ ẩm để phục vụ chức năng điều khiển máy phun sương.

Đối với dataQueue, nếu hàng đợi đã đầy, chương trình loại bỏ mẫu cũ nhất rồi gửi mẫu mới vào. Trong khi đó, xQueueOverwrite() được sử dụng với dataAlert để luôn cập nhật giá trị độ ẩm mới nhất.

Hàm vTaskDelayUntil() giúp duy trì chu kỳ đọc cảm biến ổn định ở mức 2 giây.

### d) Quá trình đóng gói và gửi Telemetry lên MQTT Broker

Task_Transmit trên Core 0 đảm nhiệm việc nhận dữ liệu từ dataQueue, đóng gói thành JSON và gửi lên MQTT Broker.

```
if (xQueueReceive(
    dataQueue,
    &data,
    pdMS_TO_TICKS(100)
  ) == pdTRUE) {
 StaticJsonDocument<128> doc;
 doc["temperature"] = data.temperature;
 doc["humidity"] = data.humidity;
 doc["sampleTime"] = data.sampleTime;
 char payload[128];
 serializeJson(doc, payload, sizeof(payload));
 mqttClient.publish(TOPIC_TELEMETRY, payload);
}
```

Đầu tiên, Task_Transmit lấy một mẫu dữ liệu từ dataQueue. Sau đó, chương trình tạo đối tượng JSON và đưa ba thông tin gồm nhiệt độ, độ ẩm và thời điểm lấy mẫu vào bản tin.

Hàm serializeJson() chuyển dữ liệu từ đối tượng JSON thành chuỗi ký tự để truyền qua MQTT. Cuối cùng, hàm mqttClient.publish() gửi bản tin lên Topic Telemetry đã khai báo.

## 6.3. Quá trình điều khiển và cập nhật trạng thái thiết bị

### a) Điều khiển máy bơm và quạt từ Website

Website cho phép người dùng bật hoặc tắt máy bơm và quạt thông qua hai nút điều khiển. Khi người dùng thao tác, Website gửi một bản tin JSON đến MQTT Broker theo Topic tương ứng.

Dữ liệu điều khiển có dạng:

{"state": true}

Trong đó, true tương ứng với bật thiết bị và false tương ứng với tắt thiết bị.

ESP32 sử dụng hàm mqttCallback() để xử lý các bản tin nhận được:

```
void mqttCallback(
  char* topic,
  byte* payload,
  unsigned int length
) {
  StaticJsonDocument<128> doc;
  DeserializationError error =
    deserializeJson(doc, payload, length);
  if (error) {
    return;
  }
  if (!doc["state"].is<bool>()) {
    return;
  }
  bool requestedState = doc["state"].as<bool>();
  if (strcmp(topic, TOPIC_PUMP) == 0) {
    setPumpState(requestedState);

  } else if (strcmp(topic, TOPIC_FAN) == 0) {
    setFanState(requestedState);
```

```
  }
  publishActuatorStatus();
}
```

Đầu tiên, chương trình sử dụng deserializeJson() để phân tích bản tin nhận được. Nếu dữ liệu JSON không hợp lệ hoặc không chứa giá trị Boolean tại trường state, chương trình sẽ bỏ qua lệnh.

Tiếp theo, ESP32 kiểm tra Topic của bản tin để xác định thiết bị cần điều khiển. Nếu bản tin thuộc Topic máy bơm, chương trình gọi setPumpState(); nếu thuộc Topic quạt, chương trình gọi setFanState().

Hai hàm này cập nhật trạng thái thiết bị và điều khiển chân GPIO tương ứng:

```
digitalWrite(
  PUMP_PIN,
  state ? HIGH : LOW
```

Tương tự, quạt được điều khiển thông qua GPIO19. Khi trạng thái thay đổi, chương trình đặt statusDirty = true để báo hiệu cần gửi trạng thái mới lên Broker.

**b) Cập nhật trạng thái thiết bị lên MQTT Broker**

Sau khi nhận và xử lý lệnh điều khiển, ESP32 cần gửi trạng thái hiện tại của các thiết bị lên MQTT Broker để Website có thể cập nhật giao diện.

Chức năng này được thực hiện thông qua hàm publishActuatorStatus():

```
StaticJsonDocument<128> doc;
doc["pump"] = pump;
doc["fan"] = fan;
doc["mistPercent"] = mist;
char payload[128];
serializeJson(doc, payload, sizeof(payload));
mqttClient.publish(
```

```
  TOPIC_STATUS,
  payload,
  true
);
```

Trước khi tạo bản tin, chương trình đọc các biến trạng thái pumpState, fanState và mistPercent dưới sự bảo vệ của actuatorMutex. Sau đó, các giá trị được đóng gói thành JSON và gửi lên Topic Status.

Tham số true trong lệnh publish() cho biết bản tin được gửi theo chế độ Retained. Điều này cho phép Broker lưu lại bản tin trạng thái cuối cùng để gửi cho các thiết bị đăng ký Topic khi chúng kết nối hoặc đăng ký nhận dữ liệu.

Nhờ cơ chế này, Website không chỉ gửi lệnh điều khiển mà còn có thể nhận lại trạng thái thực tế mà ESP32 đang quản lý.

**c) Điều khiển tự động máy phun sương theo độ ẩm**

Khác với máy bơm và quạt được điều khiển thủ công từ Website, máy phun sương hoạt động tự động dựa trên giá trị độ ẩm do cảm biến DHT22 cung cấp. Chức năng này được thực hiện bởi Task_Alert trên Core 1.

Trước tiên, chương trình sử dụng hàm getMistSetting() để xác định mức công suất và thời gian phun tương ứng:

```
if (humidity >= 40.0) {
  percent = 0;
  duration = 0;
  return false;
}
if (humidity >= 35.0) {
  percent = 25;
  duration = 10000;
```

```
} else if (humidity >= 30.0) {
  percent = 50;
  duration = 20000;
} else {
  percent = 100;
  duration = 30000;
}
return true;
```

Các ngưỡng điều khiển được thiết lập như sau:

| **Độ ẩm môi trường** | **Công suất phun** | **Thời gian phun** |
|---|---|---|
| Từ 40% trở lên | 0% (Tắt) | 0 giây |
| Từ 35% đến dưới 40% | 25% | 10 giây |
| Từ 30% đến dưới 35% | 50% | 20 giây |
| Dưới 30% | 100% | 30 giây |

Sau khi xác định mức phun, chương trình sử dụng hàm setMistPercent() để điều khiển công suất thông qua PWM:

```
void setMistPercent(uint8_t percent) {
  if (percent > 100) {
    percent = 100;
  }
  int duty = (percent * 255) / 100;
  analogWrite(MIST_PIN, duty);
  if (xSemaphoreTake(
```

```
      actuatorMutex,
      pdMS_TO_TICKS(100)
    ) == pdTRUE) {
  mistPercent = percent;
  statusDirty = true;
  xSemaphoreGive(actuatorMutex);
 }
}
```

Giá trị công suất từ 0–100% được chuyển đổi sang mức PWM từ 0–255. Ví dụ, mức 25% tương ứng với giá trị PWM xấp xỉ 63; mức 50% tương ứng với 127 và mức 100% tương ứng với 255.

Trong quá trình phun, Task_Alert tiếp tục kiểm tra độ ẩm mới từ dataAlert. Nếu độ ẩm đạt từ 40% trở lên, máy phun sương được tắt ngay. Nếu độ ẩm chuyển sang một khoảng điều khiển khác, chương trình cập nhật công suất và thời gian phun tương ứng. Khi hết thời gian phun, hệ thống đưa PWM về 0 để tắt máy phun sương.

Cơ chế này giúp hệ thống tự động điều chỉnh hoạt động của máy phun sương theo điều kiện môi trường mà không cần người dùng thao tác trực tiếp.

# KẾT LUẬN

Sau quá trình tìm hiểu, thiết kế và mô phỏng, đề tài đã xây dựng được hệ thống giám sát môi trường vườn cây công nghiệp sử dụng ESP32, FreeRTOS và giao thức MQTT. Hệ thống thực hiện được các chức năng chính gồm đọc dữ liệu nhiệt độ, độ ẩm từ cảm biến DHT22, truyền dữ liệu lên giao diện Website thông qua MQTT, điều khiển máy bơm và quạt bằng thao tác thủ công, đồng thời tự động điều chỉnh hệ thống phun sương dựa trên các ngưỡng độ ẩm đã thiết lập.

Bên cạnh đó, việc sử dụng FreeRTOS với các Task, Queue và Mutex giúp phân chia công việc hợp lý, hỗ trợ các chức năng thu thập dữ liệu, truyền thông và điều khiển hoạt động đồng thời. Qua quá trình thực hiện, em đã hiểu rõ hơn về cách tổ chức chương trình đa nhiệm trên ESP32 cũng như vai trò của MQTT trong việc trao đổi dữ liệu giữa thiết bị IoT và giao diện giám sát.

Mặc dù hệ thống đã đáp ứng được các chức năng cơ bản của đề tài, vẫn còn một số hướng có thể phát triển thêm như bổ sung biểu đồ theo dõi dữ liệu theo thời gian, lưu trữ lịch sử đo đạc và triển khai trên phần cứng thực tế để đánh giá độ ổn định.

Thông qua bài tập, em đã củng cố được kiến thức chuyên môn và rèn luyện kỹ năng thiết kế, lập trình, mô phỏng một hệ thống IoT. Em xin chân thành cảm ơn Thầy Võ Quang Sơn đã hướng dẫn và hỗ trợ trong quá trình học tập, thực hiện đề tài.

# TÀI LIỆU THAM KHẢO


- FreeRTOS Documentation: https://www.freertos.org/Documentation/
- Espressif Systems, ESP-IDF FreeRTOS: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos.html
- PubSubClient Library: https://github.com/knolleary/pubsubclient
- ArduinoJson Documentation: https://arduinojson.org/
- EMQX Public MQTT Broker: https://www.emqx.com/en/mqtt/public-mqtt5-broker
- Adafruit DHT Sensor Library: https://github.com/adafruit/DHT-sensor-library